

فعالیت کلاسی هفته‌ی هشتم - روز اول

مأموریت «از یک نورون تا یک شبکه‌ی یادگیرنده»

موضوع: مقدمه‌ای بر Deep Learning و Neural Networks

زبان برنامه‌نویسی: Python :

کتابخانه‌ها: NumPy، Scikit-learn، Matplotlib

مدت فعالیت: ۱۲۰ دقیقه

هدف فعالیت

در پایان این فعالیت، انتظار می‌رود بتوانید:

- ارتباط AI، Machine Learning، Deep Learning و Neural Networks را توضیح دهید.
- محدودیت مدل‌های کلاسیک را تشخیص دهید.
- ارتباط Logistic Regression و یک نورون مصنوعی را توضیح دهید.
- نقش Weight، Bias، Weighted Sum و Activation Function را تحلیل کنید.
- احتمال خروجی یک نورون را محاسبه کرده و با Threshold به کلاس تبدیل کنید.
- Cross-Entropy را برای یک نمونه و یک مجموعه داده محاسبه و تفسیر کنید.
- دلیل مناسب بودن Cross-Entropy برای Logistic Regression را توضیح دهید.
- مفهوم Cross-Validation را اجرا و نتایج Foldها را تحلیل کنید.
- محدودیت Perceptron و مسئله‌ی XOR را توضیح دهید.
- اهمیت Non-linearity و توابع فعال‌سازی را درک کنید.
- یک شبکه‌ی دولایه را با NumPy پیاده‌سازی کنید.

قوانین کار گروهی

برای هر مأموریت:

1. ابتدا بدون اجرای کد، خروجی را پیش‌بینی کنید.
2. هر عضو گروه باید دلیل پیش‌بینی خود را بیان کند.
3. سپس کد را اجرا کنید.
4. اگر نتیجه با پیش‌بینی شما متفاوت بود، علت تفاوت را بنویسید.
5. فقط نمایش خروجی کافی نیست؛ باید بتوانید خروجی را تفسیر کنید.

زمان‌بندی فعالیت

امتیاز	زمان	عنوان مأموریت	شماره
۵	۸ دقیقه	نقشه‌ی خانوادگی هوش مصنوعی	۱
۱۲	۱۴ دقیقه	کالبدشکافی یک نورون تصمیم‌گیر	۲
۱۸	۲۰ دقیقه	آزمایشگاه Cross-Entropy	۳
۱۲	۱۴ دقیقه	پرونده‌ی مرزهای غیرممکن	۴
۱۸	۲۰ دقیقه	کارخانه‌ی شبکه‌ی دولایه با NumPy	۵
۱۵	۱۸ دقیقه	مسابقات Cross-Validation	۶

مأموریت ۱ - نقشه‌ی خانوادگی هوش مصنوعی

زمان: ۸ دقیقه - امتیاز: ۵

بخش اول: ساخت زنجیره

مفاهیم زیر را از عمومی‌ترین تا تخصصی‌ترین مرتب کنید:

Neural Networks

Machine Learning

سپس برای هر مفهوم، یک تعریف یکجمله‌ای با زبان خودتان بنویسید.

بخش دوم: مدل‌ها و محدودیت‌هایشان

هر مدل را به محدودیت یا ویژگی مناسب متصل کنید.

مدل‌ها

1. Linear Regression

2. Logistic Regression

3. K-Nearest Neighbors

4. Decision Tree

5. Neural Network

ویژگی‌ها و محدودیت‌ها

- ممکن است به سادگی Overfit شود.
 - در زمان پیش‌بینی روی داده‌های بزرگ پرهزینه است.
 - فقط روابط خطی را یاد می‌گیرد.
 - مرز تصمیم آن خطی است و برای الگوهای پیچیده محدودیت دارد.
 - می‌تواند تبدیل‌های پیچیده یاد بگیرد، اما معمولاً به داده و محاسبات بیشتری نیاز دارد.
-

بخش سوم: انسان یا شبکه؟

دو روش زیر را مقایسه کنید:

روش سنتی:

Image → Human-designed Features → Classifier

روش: Deep Learning

Image → Neural Network → Learned Features → Prediction

در حداکثر سه جمله پاسخ دهید:

چرا یادگیری خودکار ویژگی‌ها می‌تواند در تصویر، صدا و متن از طراحی دستی ویژگی‌ها مؤثرتر باشد؟

مأموریت ۲ - کالبدشکافی یک نرون تصمیم‌گیر

زمان: ۱۴ دقیقه - امتیاز: ۱۲

یک نرون قرار است احتمال قبولی یک دانشجو را پیش‌بینی کند.

```
import numpy as np
```

```
x = np.array([
```

```
    0.8, # Study Hours
```

```
    0.9, # Attendance
```

```
    0.4 # Missed Assignments
```

```
])
```

```
w = np.array([
```

```
    1.2,
```

```
    0.6,
```

```
    -1.5
```

```
])
```

$$b = -0.5$$

مدل ابتدا مقدار زیر را محاسبه می‌کند:

$$z = w^T x + b$$

سپس:

$$p = \text{sigmoid}(z)$$

بخش اول: اثر هر ورودی

بدون استفاده از `np.dot`، سهم هر ورودی را جداگانه محاسبه کنید:

$$w_1 \times x_1$$

$$w_2 \times x_2$$

$$w_3 \times x_3$$

سپس پاسخ دهید:

1. کدام ویژگی بیشترین تأثیر را دارد؟
2. چرا وزن تعداد تکالیف انجام نشده منفی است؟
3. آیا بزرگ‌ترین وزن همیشه بیشترین تأثیر واقعی را دارد، یا مقدار Feature نیز مهم است؟
4. قدم‌مطلق وزن چه اطلاعاتی به ما می‌دهد؟

بخش دوم: محاسبه‌ی خروجی نورون

تابع Sigmoid را بنویسید:

```
def sigmoid(z):
```

```
    # کد شما
```

```
    pass
```

سپس z و Probability را محاسبه کنید.

قانون تصمیم:

Probability $\geq 0.5 \rightarrow$ Class 1

Probability $< 0.5 \rightarrow$ Class 0

بخش سوم: ساخت تابع وضعیت طبقه‌بندی

تابع زیر را کامل کنید:

```
def neuron_state(x, w, b, threshold=0.5):
```

```
    """
```

وضعیت کامل تصمیم نوروں را برمی‌گرداند.

```
    """
```

```
    # کد شما
```

```
    pass
```

خروجی تابع باید یک Dictionary با قالب زیر باشد:

```
{
```

```
    "weighted_sum": ...,
```

```
    "probability": ...,
```

```
    "threshold": ...,
```

```
    "predicted_class": ...,
```

```
    "decision": "Pass" or "Fail"
```

```
}
```

بخش چهارم: آزمایش قدرت Bias

تابع را با سه Bias زیر اجرا کنید:

bias_values = [-1.5, -0.5, 0.5]

جدولی با ستون‌های زیر بسازید:

Bias Weighted Sum Probability Predicted Class

سپس توضیح دهید:

1. آیا Bias اهمیت یک Feature خاص را تغییر داد؟
2. Bias چگونه کل تصمیم مدل را جابه‌جا کرد؟
3. چرا بدون Bias ، مرز تصمیم مجبور است از مبدأ عبور کند؟
4. چه شباهتی میان این نوروں و Logistic Regression وجود دارد؟

مأموریت ۳ - دادگاه Cross-Entropy

زمان پیشنهادی: ۲۰ دقیقه - ۱۸ امتیاز

در این سؤال چهار پیش‌بینی مختلف در دادگاه حاضر شده‌اند. شما باید مشخص کنید کدام پیش‌بینی بهتر و کدام خطرناک‌تر است.

فرمول Binary Cross-Entropy برای یک نمونه:

$$L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

در این فرمول:

- y برچسب واقعی است.
- p احتمال پیش‌بینی شده برای کلاس ۱ است.

پرونده‌های دادگاه

احتمال پیش‌بینی‌شده برای کلاس ۱ y واقعی پرونده		
A	1	0.95
B	1	0.55
C	0	0.05
D	0	0.90

بخش اول: پیش‌بینی بدون محاسبه

قبل از نوشتن کد، پرونده‌ها را از کمترین Loss تا بیشترین Loss مرتب کنید.

دلیل ترتیب خود را با هم‌گروهی‌تان مطرح کنید.

بخش دوم: پیاده‌سازی Cross-Entropy

تابع زیر را کامل کنید:

```
import numpy as np
```

```
def binary_cross_entropy(y, p):
```

```
    epsilon = 1e-12
```

```
    # برای جلوگیری از  $\log(0)$ 
```

```
    p = np.clip(p, epsilon, 1 - epsilon)
```

```
    # کد شما
```

```
    pass
```


Loss هر چهار پرونده را محاسبه کرده و در یک جدول نمایش دهید.

بخش سوم: تفسیر دو حالت اصلی

فرمول Cross-Entropy را برای دو حالت زیر ساده کنید:

وقتی برچسب واقعی برابر ۱ است

$$y = 1$$

$$L = ?$$

وقتی برچسب واقعی برابر ۰ است

$$y = 0$$

$$L = ?$$

سپس توضیح دهید:

1. وقتی $y=1$ است، مدل باید p را به چه عددی نزدیک کند؟

2. وقتی $y=0$ است، مدل باید p را به چه عددی نزدیک کند؟

3. چرا پیش‌بینی اشتباه و مطمئن باید Loss بسیار بزرگی دریافت کند؟

بخش چهارم Loss: کل داده‌ها

تابعی بنویسید که دو آرایه‌ی y_true و y_pred را دریافت کرده و میانگین Cross-Entropy را محاسبه کند.

```
def dataset_bce(y_true, y_pred):
```

```
    # کد شما
```

```
    pass
```

تابع را با داده‌های زیر آزمایش کنید:

```
y_true = np.array([1, 1, 0, 0])
```

```
y_pred = np.array([0.95, 0.55, 0.05, 0.90])
```

بخش پنجم Cross-Entropy: در برابر MSE

تابع MSE را نیز پیاده‌سازی کنید:

```
def mse_loss(y, p):
```

```
    return (y - p) ** 2
```

برای پرونده‌های A تا D، مقدار MSE و Cross-Entropy را کنار هم قرار دهید.

سپس پاسخ دهید:

1. کدام Loss پیش‌بینی اشتباه و مطمئن را شدیدتر جریمه می‌کند؟
 2. چرا Cross-Entropy برای Binary Classification مناسب‌تر است؟
 3. چرا ترکیب Logistic Regression و MSE ممکن است یک Loss غیرمحدب ایجاد کند؟
 4. چرا MSE برای Linear Regression محدب است؟
 5. عبارت $y - p$ در گرادین Cross-Entropy چه چیزی درباره‌ی جهت خطای مدل نشان می‌دهد؟
- پاسخ قسمت‌های سوم تا پنجم می‌تواند کوتاه باشد.

مأموریت ۴ - پرونده‌ی مرزهای غیرممکن

زمان: ۱۴ دقیقه - امتیاز: ۱۲

بخش اول: مرز تصمیم Perceptron

Perceptron ابتدا محاسبه می‌کند:

$$z = w^T x + b$$

و سپس از Step Function استفاده می‌کند:

Class 1 → باشد $z \geq 0$ اگر

Class 0 → باشد $z < 0$ اگر

به پرسش‌های زیر پاسخ دهید:

1. معادله‌ی مرز تصمیم چیست؟
2. در داده‌ی یک‌بعدی، مرز تصمیم چه شکلی دارد؟
3. در داده‌ی دوبعدی، مرز تصمیم چه شکلی دارد؟
4. در داده‌ی سه‌بعدی، مرز تصمیم چه شکلی دارد؟
5. در فضای n بعدی، مرز تصمیم چه نام دارد؟
6. Weight ها جهت و شیب مرز را چگونه تغییر می‌دهند؟
7. Bias موقعیت مرز تصمیم را چگونه تغییر می‌دهد؟

بخش دوم: معمای XOR

جدول: XOR

x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0

در دو یا سه جمله توضیح دهید:

چرا یک Perceptron نمی‌تواند XOR را حل کند؟

پاسخ شما باید شامل عبارت‌های زیر باشد:

Linearly Separable

Single Decision Boundary

Weights and Bias

بخش سوم: دو لایه‌ی خطی، اما فقط یک خط!

دو تابع زیر را در نظر بگیرید:

```
def layer_1(x):
```

```
    return 2 * x + 1
```

```
def layer_2(h):
```

```
    return -3 * h + 4
```

خروجی نهایی:

```
output = layer_2(layer_1(x))
```

وظایف:

1. عبارت نهایی را ساده کنید.
2. یک تابع خطی معادل بنویسید.
3. برای $x = [-2, -1, 0, 1, 2]$ بررسی کنید که دو روش خروجی یکسان دارند.
4. توضیح دهید چرا قراردادن چند لایه‌ی خطی پشت سر هم، همچنان فقط یک تابع خطی ایجاد می‌کند.

بخش چهارم: ورودی Non-linearity

تابع اول را تغییر دهید:

```
def relu(x):
```

```
return max(0, x)
```

```
def layer_1(x):
```

```
    return relu(2 * x + 1)
```

خروجی شبکه را برای مقادیر زیر محاسبه کنید:

```
x_values = [-2, -1, 0, 1, 2]
```

سپس توضیح دهید چرا اکنون رفتار کلی شبکه را نمی‌توان در تمام ورودی‌ها با یک خط واحد نمایش داد.

بخش پنجم: انتخاب Activation Function

ورودی زیر را در نظر بگیرید:

```
z = np.array([-5, -1, 0, 1, 5], dtype=float)
```

اول: پیاده‌سازی

توابع زیر را با NumPy بنویسید:

```
def sigmoid(z):
```

```
    pass
```

```
def relu(z):
```

```
    pass
```

```
def tanh_activation(z):
```

```
    pass
```

راهنمایی:

$$\text{Sigmoid}(z) = 1 / (1 + \exp(-z))$$

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{Tanh}(z) = \tanh(z)$$

خروجی هر تابع را برای آراییه z نمایش دهید.

دوم: جدول مقایسه

جدولی با ستون‌های زیر ایجاد کنید:

z Sigmoid Tanh ReLU

سپس پاسخ دهید:

1. خروجی Sigmoid در چه بازه‌ای قرار دارد؟

2. خروجی Tanh در چه بازه‌ای قرار دارد؟

3. چرا گفته می‌شود Tanh خروجی Zero-centered دارد؟

4. ReLU با اعداد منفی چه می‌کند؟

5. برای اعداد مثبت، ReLU چه رفتاری دارد؟

سوم: انتخاب تابع مناسب

برای هر مسئله، تابع فعال‌سازی خروجی مناسب را انتخاب کنید:

انتخاب شما	نوع مسئله
؟	Binary Classification
؟	Multi-Class Classification

Regression	؟
------------	---

گزینه‌ها:

Sigmoid

Softmax

Linear

چهارم: مشکلات فعال‌سازی‌ها

هر مورد را حداکثر در دو جمله توضیح دهید:

1. Vanishing Gradient در Sigmoid یا Tanh چیست؟

2. Dead Neuron در ReLU چیست؟

3. GELU بیشتر در چه خانواده‌ای از مدل‌ها استفاده می‌شود؟

4. چرا معمولاً برای لایه‌ی مخفی از ReLU و برای خروجی Binary Classification از Sigmoid استفاده می‌کنیم؟

مأموریت ۵ - کارخانه‌ی شبکه‌ی دولایه با NumPy

زمان: ۲۰ دقیقه - امتیاز: ۱۸

قرار است یک شبکه‌ی دولایه بسازید که سه ویژگی را دریافت کند و در پایان عددی بین صفر و یک تولید کند.

معماری:

3 Input Features

↓

Hidden Layer: 2 Neurons

↓

ReLU



Output Layer: 1 Neuron



Sigmoid



Probability



Threshold



Class 0 or Class 1

پارامترهای شبکه:

```
import numpy as np
```

```
W1 = np.array([  
    [ 0.7, -0.4,  0.8],  
    [-0.6,  0.9,  0.2]  
])
```

```
b1 = np.array([0.1, -0.2])
```

```
W2 = np.array([1.2, -0.8])
```


b2 = -0.3

ورودی‌ها:

x_A = np.array([0.6, -1.0, 0.8])

x_B = np.array([-0.8, 0.7, -0.3])

بخش اول: پیش‌بینی Shape ها

Shape هر متغیر را قبل از اجرا مشخص کنید:

x

W1

b1

z1

a1

W2

b2

z2

probability

سپس توضیح دهید چرا تعداد ستون‌های W1 باید برابر تعداد Features های ورودی باشد.

بخش دوم: ساخت نرون مخفی

def relu(z):

 return np.maximum(0, z)

```
def hidden_layer(x, W, b):
```

```
    """
```

```
    z و خروجی فعال شده ی لایه ی مخفی را برمی گرداند.
```

```
    """
```

```
    # کد شما
```

```
    pass
```

```
    خروجی پیشنهادی:
```

```
    return z, activation
```

بخش سوم: ساخت نورون خروجی

```
def sigmoid(z):
```

```
    return 1 / (1 + np.exp(-z))
```

```
def output_layer(hidden_output, W, b):
```

```
    """
```

```
    z و Probability کلاس 1 را برمی گرداند.
```

```
    """
```

```
    # کد شما
```

pass

بخش چهارم: اتصال توابع مانند یک شبکه

تابع زیر را تکمیل کنید:

```
def classification_network(
```

```
    x,
```

```
    W1,
```

```
    b1,
```

```
    W2,
```

```
    b2,
```

```
    threshold=0.5
```

```
):
```

```
    """
```

ورودی را از دو لایه عبور می‌دهد و

وضعیت کامل طبقه‌بندی را برمی‌گرداند.

```
    """
```

```
    # مرحله hidden_layer1 :
```

```
    # مرحله output_layer2 :
```

```
    # مرحله probability → class3 :
```

```
    # مرحله 4: ساخت dictionary
```

pass

خروجی تابع باید به شکل زیر باشد:

```
{  
  "hidden_weighted_sum": ...,  
  "hidden_activation": ...,  
  "output_weighted_sum": ...,  
  "probability_class_1": ...,  
  "threshold": ...,  
  "predicted_class": ...,  
  "classification_state": "Confident Class 0",  
                          "Uncertain",  
                          or  
                          "Confident Class 1"  
}
```

برای تعیین وضعیت، از قانون زیر استفاده کنید:

اگر: $\text{Probability} < 0.30$

Confident Class 0

اگر $0.30 \leq \text{Probability} < 0.70$

Uncertain

اگر: $\text{Probability} \geq 0.70$

Confident Class 1

کلاس نهایی همچنان باید با threshold تعیین شود.

بخش پنجم: آزمایش شبکه

شبکه را برای x_A و x_B اجرا کنید.

جدولی با ستون‌های زیر بسازید:

Input Hidden Output Probability Threshold Class State

پیش از اجرا حدس بزنید کدام ورودی Probability بیشتری خواهد داشت.

بخش ششم: مهندسی تصمیم

برای یک ورودی ثابت، شبکه را با Threshold های زیر اجرا کنید:

thresholds = [0.3, 0.5, 0.8]

پاسخ دهید:

1. آیا Probability با تغییر Threshold عوض شد؟

2. آیا Predicted Class عوض شد؟

3. Threshold بخشی از محاسبات لایه‌های شبکه است یا یک قانون تصمیم پس از تولید Probability؟

4. چرا در بعضی کاربردها مانند تشخیص بیماری ممکن است Threshold برابر ۰/۵ بهترین انتخاب نباشد؟

مأموریت ۶ – مسابقات Cross-Validation

زمان: ۱۸ دقیقه – امتیاز: ۱۵

در این مأموریت نباید از Linear Regression استفاده کنید.

قرار است چند Decision Tree Classifier در یک مسابقه‌ی پنج‌مرحله‌ای شرکت کنند.

از Iris Dataset کامل و هر سه کلاس استفاده کنید.

بخش اول: آماده‌سازی داده

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

Shape داده‌ها و تعداد کلاس‌ها را چاپ کنید.

بخش دوم: یک تقسیم، یک نتیجه

مدل زیر را با یک Train/Test Split ارزیابی کنید:

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
stratify=y
)

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)
```

Accuracy مدل را روی Test Set محاسبه کنید.

سپس پاسخ دهید:

آیا یک Accuracy حاصل از یک Split برای قضاوت قطعی درباره‌ی مدل کافی است؟

بخش سوم: مسابقه‌ی پنج Fold

از StratifiedKFold و cross_val_score استفاده کنید:

```
from sklearn.model_selection import (
    StratifiedKFold,
    cross_val_score
)
```

```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
```

)

مدل زیر را با پنج Fold ارزیابی کنید:

```
model = DecisionTreeClassifier(
```

```
    max_depth=3,
```

```
    random_state=42
```

)

موارد زیر را نمایش دهید:

Fold هر Accuracy

Mean Accuracy

Standard Deviation

Minimum Accuracy

Maximum Accuracy

بخش چهارم: انتخاب قهرمان

مدلهایی با max_depth های زیر را مقایسه کنید:

```
depth_values = [1, 2, 3, 4, None]
```

برای هر مقدار، موارد زیر را محاسبه کنید:

Mean CV Accuracy

Standard Deviation

نتایج را در یک DataFrame قرار دهید:

```
max_depth mean_accuracy std_accuracy
```


سپس یک مدل را به عنوان قهرمان انتخاب کنید.

انتخاب شما نباید فقط بر اساس بیشترین میانگین باشد؛ پایداری مدل در Fold های مختلف را نیز در نظر بگیرید.

بخش پنجم: گفت و گوی داوران

به پرسش های زیر پاسخ دهید:

1. Cross-Validation چه مشکلی را در مقایسه با یک Train/Test Split کاهش می دهد؟
 2. در 5-Fold Cross-Validation، هر نمونه چند بار در Validation قرار می گیرد؟
 3. هر نمونه چند بار در Training قرار می گیرد؟
 4. چرا برای Classification استفاده از StratifiedKFold مفید است؟
 5. Mean Accuracy چه چیزی را نشان می دهد؟
 6. Standard Deviation چه چیزی را نشان می دهد؟
 7. مدلی با میانگین 0.96 و انحراف معیار 0.01 بهتر است یا مدلی با میانگین 0.97 و انحراف معیار 0.09؟ پاسخ قطعی نیست؛ استدلال کنید.
 8. چرا نباید Test Set نهایی را بارها برای انتخاب `max_depth` استفاده کنیم؟
 9. تفاوت Cross-Validation با آموزش همزمان پنج مدل برای استفاده در محصول چیست؟
 10. پس از انتخاب بهترین Hyperparameter با Cross-Validation، مدل نهایی را روی چه داده ای آموزش می دهیم؟
-

بخش ششم: چالش کشف خطا

کد زیر را بررسی کنید:

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
X,  
y,  
test_size=0.2,  
random_state=42  
)
```

for depth in [1, 2, 3, 4, None]:

```
model = DecisionTreeClassifier(  
    max_depth=depth,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

```
score = model.score(X_test, y_test)
```

```
print(depth, score)
```

پاسخ دهید:

چرا استفاده‌ی مکرر از همان Test Set برای انتخاب بهترین max_depth می‌تواند باعث خوش‌بینی بیش‌ازحد در ارزیابی نهایی شود؟

نسخه‌ی صحیح فرایند را به‌صورت یک نمودار متنی بنویسید:

↓

Cross-Validation

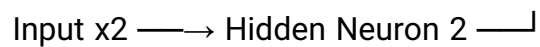
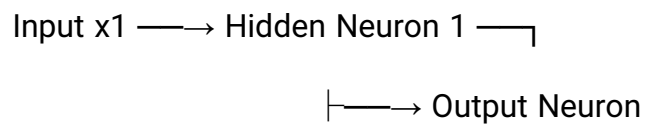
↓

Final Training

One-Time Test Evaluation

مأموریت 7 - سه نگهبان دروازه‌ی OR

عملگر منطقی OR را با دقتاً سه نورون پیاده‌سازی کنید:



تابع فعال سازی:

```
def step(z):  
    return 1 if z >= 0 else 0
```

تابع یک: Perceptron

```
def perceptron(x, w, b):
```

$$z = \text{np.dot}(w, x) + b$$

```
return step(z)
```

ساختار شبکه:

```
def three_neuron_or(x1, x2):
```

```
    inputs = np.array([x1, x2])
```

```
    h1 = perceptron(
```

```
        inputs,
```

```
        w_h1,
```

```
        b_h1
```

```
)
```

```
    h2 = perceptron(
```

```
        inputs,
```

```
        w_h2,
```

```
        b_h2
```

```
)
```

```
    hidden_outputs = np.array([h1, h2])
```

```
    output = perceptron(
```

```

hidden_outputs,

w_output,

b_output

)

```

```

return output

```

وزن ها و Bias ها را خودتان پیدا کنید.

شبکه باید تمام حالت های زیر را درست حل کند:

x1	x2	OR مورد انتظار
0	0	0
0	1	1
1	0	1
1	1	1

محدودیت ها:

- استفاده از عملگر آماده ی OR مجاز نیست.
- خروجی نباید به صورت دستی Hard-code شود.
- هر سه نورون باید در تولید نتیجه نقش داشته باشند.
- مقادیر z و خروجی هر سه نورون را نمایش دهید.

در پایان توضیح دهید:

1. چرا OR با یک Perceptron نیز قابل حل است؟
2. چرا این سؤال با وجود نیاز نداشتن به سه نورون، برای درک شبکه ی چندلایه مفید است؟
3. کدام نورون مانند یک Feature Detector عمل می کند؟

آبرچالش Bonus - تبدیل شبکه‌ی OR به XOR

۴ امتیاز Bonus

شبکه‌ی سه‌نورونی سؤال قبل را تغییر دهید تا XOR را حل کند.

جدول مورد انتظار:

x1 x2 XOR

0 0 0

0 1 1

1 0 1

1 1 0

شرایط:

- از دو نورون در Hidden Layer استفاده کنید.
- از یک نورون در Output Layer استفاده کنید.
- فقط از NumPy، Weighted Sum، Bias و Step Function استفاده کنید.
- استفاده از xor، یا Hard-code کردن پاسخ ممنوع است.
- هر چهار حالت باید درست پیش‌بینی شوند.

در پایان، با زبان خودتان توضیح دهید:

چرا یک Perceptron نمی‌تواند XOR را حل کند، اما ترکیب چند Perceptron می‌تواند چند مرز تصمیم ایجاد کرده و مسئله را حل کند؟
